

1 Quicksort

- (a) Sort the following unordered list using stable Quicksort. Assume that we always choose first element as the pivot and that we use the 3-way merge partitioning process described in lecture. Show the steps taken at each partitioning step.

18, 7, 22, 34, 99, 18, 11, 4

18	7	22	34	99	18	11	4
7	11	4	18	18	22	34	99
4	7	11	18	18	22	34	99

- (b) What is the best and worst case running time of Quicksort with Hoare Partitioning on N elements? Given the two lists $[4, 4, 4, 4, 4]$ and $[1, 2, 3, 4, 5]$, assuming we pick the first element as the pivot every time, which list would happen to result in better runtime?

Best: $\Theta(N \log N)$ better

Worst: $\Theta(N^2)$

- (c) What are two techniques that can be used to reduce the probability of Quicksort taking the worst case running time?

Random choose pivots.

Shuffle the list.

2 Comparison Sorts Summary

- (a) When choosing an appropriate algorithm, there are often several trade-offs that we need to consider. Complete the chart for the following sorting algorithms: give the expected time complexity in the worst case, in the best case, and whether or not each sort is stable.

	Time Complexity (Best)	Time Complexity (Worst)	Stability	In Place
Selection Sort	n^2	n^2	No	Yes
Insertion Sort	n	n^2	Yes	Yes
Heapsort	n	$n \log n$	No	Yes
Mergesort	$n \log n$	$n \log n$	Yes	No.
Quicksort (w/ Hoare Partitioning)	$n \log n$	n^2	No	Yes.

- (b) For selection sort, give an example of a list where the order of equivalent items is not preserved.

$\emptyset$ 3a 3b 3c 1
 1 3b 3c 3a
 1 3b 3c 3a

- (c) Notice that the worst-case runtime in the comparison sorts on an N element array listed above are lower bounded by $\Theta(N \log N)$. Can there be a sort that runs faster than $\Theta(N \log N)$ in the worst-case?

Yes. Radix sort. when avoiding comparison.

3 Radix Sorts

- (a) Sort the following list using LSD Radix Sort with counting sort. Show the steps taken after each round of counting sort. The first row is the original list and the last two rounds are already filled for you.

	30395	30326	43092	30315
1	43092	30395	30315	30326
2	30315	30326	43092	30395
3	43092	30315	30326	30395
4	30315	30326	30395	43092
5	30315	30326	30395	43092

- (b) Sort the following list using MSD Radix Sort with counting sort. Show the steps taken after each round of counting sort. The first row is the original list and the first three rounds are already filled for you.

	30395	30326	43092	30315
1	<u>3</u> 0395	<u>3</u> 0326	<u>3</u> 0315	<u>4</u> 0392
2	<u>3</u> 0395	<u>3</u> 0326	<u>3</u> 0315	<u>4</u> 0392
3	<u>3</u> 0395	<u>3</u> 0326	<u>3</u> 0315	<u>4</u> 0392
4	<u>30315</u>	<u>30326</u>	<u>30395</u>	<u>40392</u>
5	30315	30326	30395	40392

- (c) Give the best case runtime, worst case runtime, and whether or not the sort is stable for both LSD and MSD radix sort. Assume we have N elements, a radix R , and a maximum number of digits in an element W .

	Time Complexity (Best)	Time Complexity (Worst)	Stability
LSD Radix Sort	$\Theta(WN + NR)$	$\Theta(WN + WR)$	Yes
MSD Radix Sort	$\Theta(N + R)$	$\Theta(WN + WR)$	Yes

- (d) We just saw above that radix sort has great runtime with respect to the number of elements in the list. Given this fact, should we say that radix sort is the best sort to use?

No, we can't. Runtime based on W and N , R .
If R big, can't use. Also not suitable for non-digit.

4 Bounding Practice *Extra*

Given an array of n elements, the heapification operation permutes the elements of the array into a heap. There are many solutions to the heapification problem. One approach is bottom-up heapification, which treats the existing array as a heap and rearranges all nodes from the bottom up to satisfy the heap invariant. Another is top-down heapification, which starts with an empty heap and inserts all elements into it.

- (a) Why can we say that any solution for heapification requires $\Omega(n)$ time?

Ω . it's the lower bound. When using bottom-up heapification.

- (b) Show that the worst-case runtime for top-down heapification is in $\Theta(n \log n)$. Why does this mean that the optimal solution for heapification takes $O(n \log n)$ time?

Top-down actually is N times insert. So. $\Theta(n \log n)$.

$O(n \log n)$ means upper bound. We use Big O to describe the worst case.

- (c) In contrast, bottom-up heapification is an $O(n)$ algorithm. Is bottom-up heapification asymptotically-optimal?

Yes. it is the best case.

- (d) Show that the running time of bottom-up heapify is $\Theta(n)$.

Some useful facts:

$$\sum_{i=0}^{\infty} x^i = \frac{1}{1-x}$$

Taking the derivative:

$$\frac{d}{dx} \left(\sum_{i=0}^{\infty} x^i \right) = \frac{1}{(1-x)^2}$$